Assignment - 14

Target URL for your POM:
https://www.demoblaze.com/index.html

Exercise 1: The "Base Page" & Chaining (Navigation)
The Task: Create a `BasePage` that contains common methods (like `click`, `find`, and `get_text`). Then, create a `HomePage` class.
What to test: Navigate to the "Laptops" category from the sidebar.
POM Logic: The `click_laptops()` method in `HomePage` should return a new instance of the `LaptopsCategoryPage`.
The Goal: Your test should look like this: `home.click_laptops().verify_laptop_list_presence()`.

Exercise 2: List Elements & Dynamic Data (Product Search)
The Task: Create a `CategoryPage` that handles the list of products displayed.
What to test: Navigate to the "Phones" category. Use `find_elements` to capture all product names displayed on the page.
POM Logic: Write a method `get_all_product_names()` that returns a list of strings.
The Goal: In your test, assert that "Samsung galaxy s6" is present in the returned list.

Exercise 3: Synchronization & Alerts (Adding to Cart)
The Task: Create a `ProductDetailsPage`.
What to test: Click on "Sony vaio i5", then click the "Add to cart" button.
POM Logic: When "Add to cart" is clicked, a browser alert (JavaScript pop-up) appears after a short delay.
The Goal: Inside your POM method `add_product_to_cart()`, implement a `WebDriverWait` for `alert_is_present()`, accept the alert, and log the action using the `logging` module.

Exercise 4: Multi-Step Flow & Component POM (Checkout)
The Task: Create a `CartPage` and a `PurchaseModal` (as a component or separate class).
What to test: Navigate to the "Cart", click "Place Order", fill in the modal form (Name, Country, City, Card, Month, Year), and click "Purchase".
POM Logic: Encapsulate the form fields within a `fill_purchase_form(data_dict)` method that takes a dictionary of info.
The Goal: Verify the "Thank you for your purchase!" success message appears. This test must trigger an Allure Step for every field filled.

Exercise 5: Error Handling & Screenshots (Negative Test)
The Task: Handle the "Login" modal.
What to test: Click "Log in" in the header, enter a valid username but a wrong password, and click "Log in".
POM Logic: Use `pytest` hooks in `conftest.py` to monitor this test.
The Goal: Since the login fails, an alert saying "Wrong password." appears. Your test should catch this.

Screenshot Challenge: Intentionally write an assertion that fails (e.g., asserting the alert text is "Success").
The `conftest.py` must automatically capture a screenshot of the login modal and attach it to the Allure Report

Solution:

Project Structure:

```
C:\Wipro Training\Assignments\
│
├── pytest.ini                  # Global settings (sets pythonpath and alluredir defaults)
│
└── assignment_14/              # Your core project directory
    │
    ├── allure-2.41.0/          # Local Allure compiler binary tool (extracted zip)
    │   └── bin/
    │       ├── allure.bat      # Executable used to generate/serve dashboards
    │       └── allure.ps1
    │
    ├── allure-results/         # Created dynamically by pytest (contains raw .json data)
    │
    ├── conftest.py             # Shared browser fixture & Allure failure screenshot hook
    │
    ├── pages/                  # Page Object Models (POM) directory
    │   ├── __init__.py
    │   ├── base_page.py        # Handles selenium waits, clicks, and text captures
    │   ├── home_page.py        # Category page logic & chaining actions
    │   ├── product_details_page.py # Alert synchronization and logging logic
    │   ├── cart_page.py        # Checkout orchestration page
    │   │
    │   └── components/         # Sub-page modal fragments
    │       ├── __init__.py
    │       ├── login_modal.py    # Login modal synchronization & interaction logic
    │       └── purchase_modal.py # Purchase form data handling & Allure step wrappers
    │
    └── tests/                  # Automation test cases
        ├── __init__.py
        ├── test_demoblaze.py     # Contains Exercises 1, 2, 3, and 4
        └── test_login.py         # Contains Exercise 5 (Intentional failure test)
```

Code Files are:

conftest.py

```python
import pytest
import allure
from selenium import webdriver

@pytest.fixture(scope="function")
def driver(request):
   options = webdriver.ChromeOptions()
   options.add_argument("--start-maximized")
   driver = webdriver.Chrome(options=options)
   driver.get("https://demoblaze.com")

   # Attach driver to the request node so the hook can access it on failure
   request.node.funcargs['driver'] = driver

   yield driver
   driver.quit()


@pytest.hookimpl(tryfirst=True, hookwrapper=True)
def pytest_runtest_makereport(item, call):
   # Execute all other hooks to obtain the report object
   outcome = yield
   rep = outcome.get_result()

   # Check if the test failed during the execution phase
   if rep.when == "call" and rep.failed:
       driver = item.funcargs.get("driver")
       if driver:
           try:
               # Capture and attach the screenshot to Allure
               screenshot = driver.get_screenshot_as_png()
               allure.attach(
                   screenshot,
                   name="Failure_Screenshot",
                   attachment_type=allure.attachment_type.PNG
               )
           except Exception as e:
               print(f"Failed to capture screenshot: {e}")
```

login_modal.py

```
from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC
from pages.base_page import BasePage

class LoginModal(BasePage):
   USERNAME_FIELD = (By.ID, "loginusername")
   PASSWORD_FIELD = (By.ID, "loginpassword")
   LOGIN_BTN = (By.XPATH, "//button[text()='Log in']")

   def login_with_credentials(self, username, password):
       # Explicitly wait until the input element is ready to accept text
       username_el =
self.wait.until(EC.element_to_be_clickable(self.USERNAME_FIELD))
       username_el.clear()
       username_el.send_keys(username)

       password_el =
self.wait.until(EC.element_to_be_clickable(self.PASSWORD_FIELD))
       password_el.clear()
       password_el.send_keys(password)

       self.click(self.LOGIN_BTN)

       # Wait for the validation or failure alert context to appear
       self.wait.until(EC.alert_is_present())
       alert = self.driver.switch_to.alert
       alert_text = alert.text
       alert.accept()
       return alert_text
```

purchase_modal.py

```
import allure
from selenium.webdriver.common.by import By
from pages.base_page import BasePage
from selenium.webdriver.support import expected_conditions as EC

class PurchaseModal(BasePage):
   NAME = (By.ID, "name")
   COUNTRY = (By.ID, "country")
   CITY = (By.ID, "city")
   CARD = (By.ID, "card")
   MONTH = (By.ID, "month")
   YEAR = (By.ID, "year")
   PURCHASE_BTN = (By.XPATH, "//button[text()='Purchase']")
```

```
    def fill_purchase_form(self, data_dict):
        self._fill_field(self.NAME, data_dict.get("name"), "Name")
        self._fill_field(self.COUNTRY, data_dict.get("country"), "Country")
        self._fill_field(self.CITY, data_dict.get("city"), "City")
        self._fill_field(self.CARD, data_dict.get("card"), "Card")
        self._fill_field(self.MONTH, data_dict.get("month"), "Month")
        self._fill_field(self.YEAR, data_dict.get("year"), "Year")
        self.click(self.PURCHASE_BTN)

    def _fill_field(self, locator, value, field_name):
        with allure.step(f"Typing '{value}' into field: {field_name}"):
            # Upgrade from presence verification to clickability verification
            element = self.wait.until(EC.element_to_be_clickable(locator))
            element.clear()
            element.send_keys(value)
```

base_page.py

```
import allure
from selenium.webdriver.common.by import By
from pages.base_page import BasePage
from selenium.webdriver.support import expected_conditions as EC

class PurchaseModal(BasePage):
    NAME = (By.ID, "name")
    COUNTRY = (By.ID, "country")
    CITY = (By.ID, "city")
    CARD = (By.ID, "card")
    MONTH = (By.ID, "month")
    YEAR = (By.ID, "year")
    PURCHASE_BTN = (By.XPATH, "//button[text()='Purchase']")

    def fill_purchase_form(self, data_dict):
        self._fill_field(self.NAME, data_dict.get("name"), "Name")
        self._fill_field(self.COUNTRY, data_dict.get("country"), "Country")
        self._fill_field(self.CITY, data_dict.get("city"), "City")
        self._fill_field(self.CARD, data_dict.get("card"), "Card")
        self._fill_field(self.MONTH, data_dict.get("month"), "Month")
        self._fill_field(self.YEAR, data_dict.get("year"), "Year")
        self.click(self.PURCHASE_BTN)

    def _fill_field(self, locator, value, field_name):
        with allure.step(f"Typing '{value}' into field: {field_name}"):
            # Upgrade from presence verification to clickability verification
            element = self.wait.until(EC.element_to_be_clickable(locator))
            element.clear()
            element.send_keys(value)
```

cart_page.py

```
from selenium.webdriver.common.by import By
from pages.base_page import BasePage
from pages.components.purchase_modal import PurchaseModal

class CartPage(BasePage):
    PLACE_ORDER_BTN = (By.XPATH, "//button[text()='Place Order']")
    SUCCESS_MARKER = (By.XPATH, "//h2[text()='Thank you for your purchase!']")

    def click_place_order(self):
        self.click(self.PLACE_ORDER_BTN)
        return PurchaseModal(self.driver)

    def get_success_message(self):
        return self.get_text(self.SUCCESS_MARKER)
```

home_page.py

```
from selenium.webdriver.common.by import By
from pages.base_page import BasePage
import time
from selenium.common.exceptions import StaleElementReferenceException

class HomePage(BasePage):
    LAPTOPS_CAT = (By.XPATH, "//a[@onclick=\"byCat('notebook')\"]")
    PHONES_CAT = (By.XPATH, "//a[@onclick=\"byCat('phone')\"]")
    PRODUCT_LINKS = (By.CLASS_NAME, "card-title")
    NAV_CART = (By.ID, "cartur")
    NAV_LOGIN = (By.ID, "login2")

    def click_laptops(self):
        self.click(self.LAPTOPS_CAT)
        return LaptopsCategoryPage(self.driver)

    def click_phones(self):
        self.click(self.PHONES_CAT)
        return CategoryPage(self.driver)

    def click_product(self, product_name):
        product_locator = (By.LINK_TEXT, product_name)
        self.click(product_locator)
        from pages.product_details_page import ProductDetailsPage
        return ProductDetailsPage(self.driver)

    def click_cart(self):
        self.click(self.NAV_CART)
        from pages.cart_page import CartPage
```

```
        return CartPage(self.driver)

    def click_login(self):
        self.click(self.NAV_LOGIN)
        from pages.components.login_modal import LoginModal
        return LoginModal(self.driver)


class CategoryPage(BasePage):
    PRODUCT_NAMES = (By.CLASS_NAME, "hrefch")

    def get_all_product_names(self):
        # Allow the dynamic DOM grid to refresh after clicking the category
link
        time.sleep(1.5)

        # Implement a retry loop to catch any rapid UI re-rendering spikes
        for _ in range(3):
            try:
                elements = self.find_all(self.PRODUCT_NAMES)
                return [el.text for el in elements if el.text]
            except StaleElementReferenceException:
                time.sleep(0.5)
                continue
        raise StaleElementReferenceException("Product list element DOM updates
failed to stabilize.")


class LaptopsCategoryPage(CategoryPage):
    def verify_laptop_list_presence(self):
        product_names = self.get_all_product_names()
        assert len(product_names) > 0, "Laptop list is empty!"
        return self


# Add this locator inside your HomePage class:


# Add this method inside your HomePage class:
```

product_details_page.py

```
import logging
from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC
from pages.base_page import BasePage
```

```
logger = logging.getLogger(__name__)

class ProductDetailsPage(BasePage):
    ADD_TO_CART_BTN = (By.XPATH, "//a[text()='Add to cart']")

    def add_product_to_cart(self):
        self.click(self.ADD_TO_CART_BTN)
        alert = self.wait.until(EC.alert_is_present())
        alert_text = alert.text
        alert.accept()
        logger.info(f"Accepted browser alert with message: {alert_text}")
        return self
```

test_demoblaze.py

```
import pytest
import logging
from selenium import webdriver
from pages.home_page import HomePage

logging.basicConfig(level=logging.INFO)


@pytest.fixture
def driver():
    options = webdriver.ChromeOptions()
    options.add_argument("--start-maximized")
    driver = webdriver.Chrome(options=options)
    driver.get("https://www.demoblaze.com/index.html")
    yield driver
    driver.quit()


def test_exercise_1_chaining(driver):
    home = HomePage(driver)
    # Execution matches the requested goal structure precisely
    home.click_laptops().verify_laptop_list_presence()


def test_exercise_2_dynamic_list(driver):
    home = HomePage(driver)
    phones_page = home.click_phones()
    product_list = phones_page.get_all_product_names()
    assert "Samsung galaxy s6" in product_list


def test_exercise_3_synchronization_alerts(driver):
    home = HomePage(driver)
```

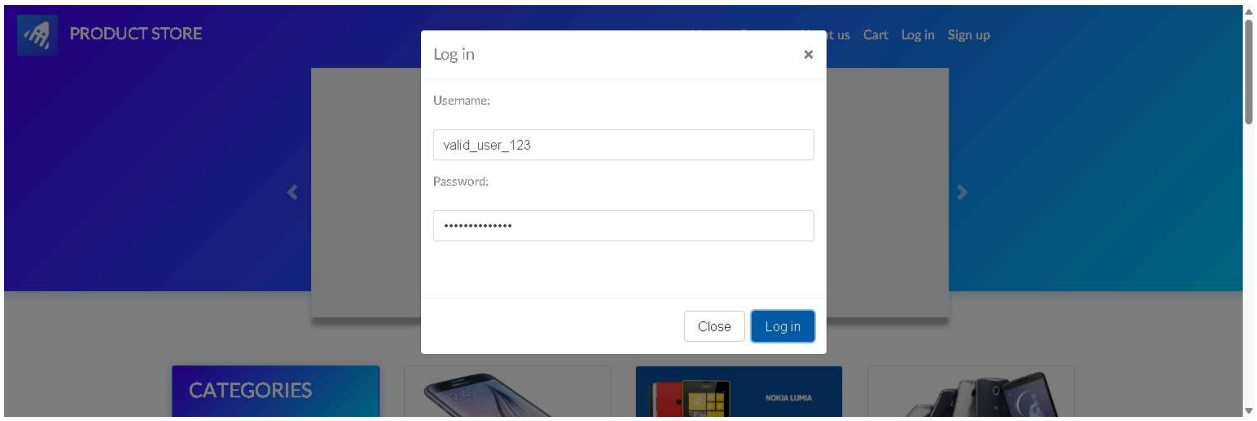
```
    product_page = home.click_product("Sony vaio i5")
    product_page.add_product_to_cart()


def test_exercise_4_checkout_flow(driver):
    home = HomePage(driver)

    # Setup step: Add item to trigger functional checkout
    product_page = home.click_product("Sony vaio i5")
    product_page.add_product_to_cart()

    cart = home.click_cart()
    modal = cart.click_place_order()

    customer_data = {
        "name": "Jane Doe",
        "country": "USA",
        "city": "Austin",
        "card": "1234567890",
        "month": "12",
        "year": "2028"
    }

    modal.fill_purchase_form(customer_data)
    assert cart.get_success_message() == "Thank you for your purchase!"
```

test_login.py

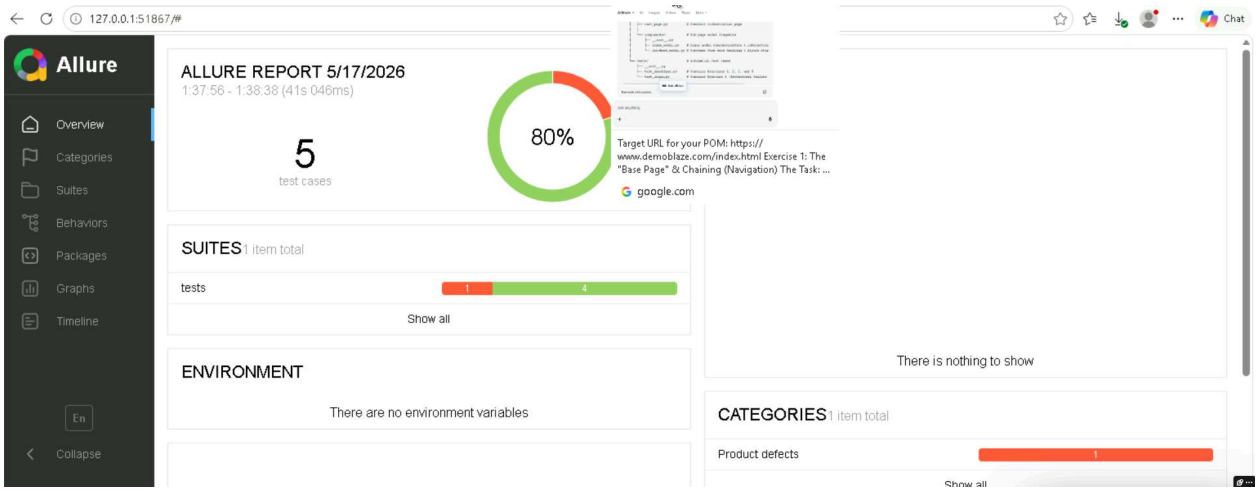
```
import pytest
from pages.home_page import HomePage


def test_exercise_5_login_failure_screenshot(driver):
    home = HomePage(driver)
    login_modal = home.click_login()

    # Step 1: Submit user handle alongside an incorrect passcode
    alert_text = login_modal.login_with_credentials("valid_user_123",
"wrong_password")

    # Step 2: Validate the actual alert message returned by Demoblaze
    assert alert_text == "Please fill out Username and Password."

    # Step 3: Trigger your intentional failure challenge to force the conftest
screenshot hook
    assert alert_text == "Success", "Intentional failure to trigger Allure
screenshot attachment."
```

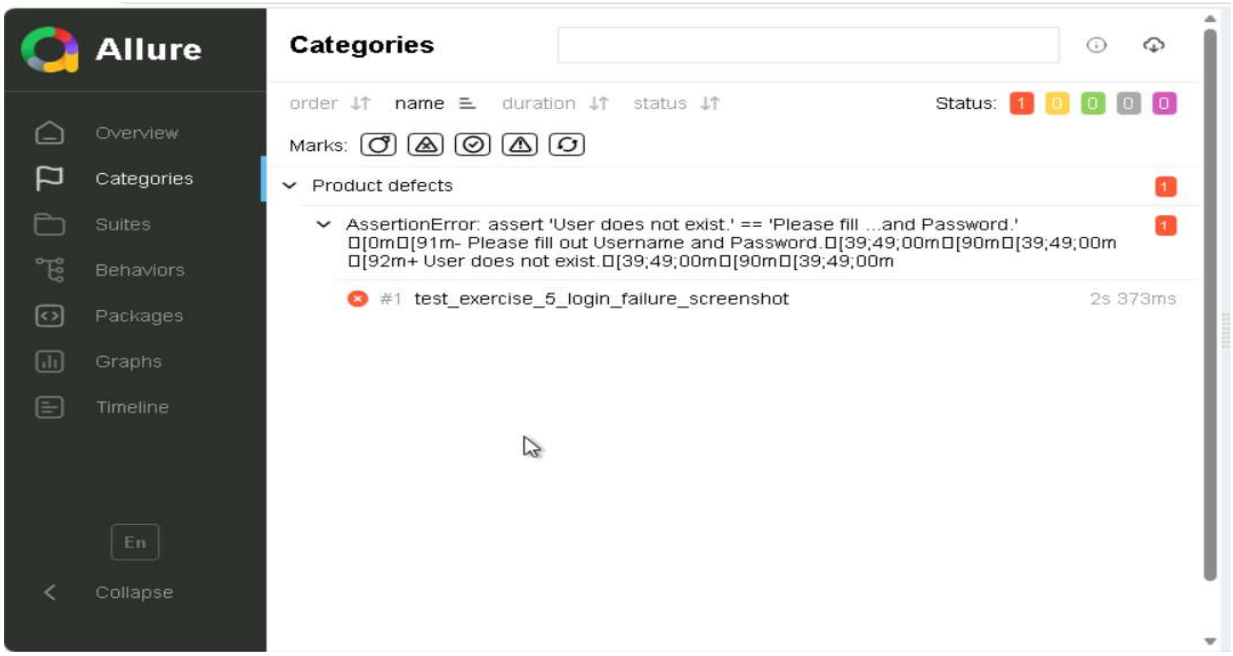
Allure Reports:

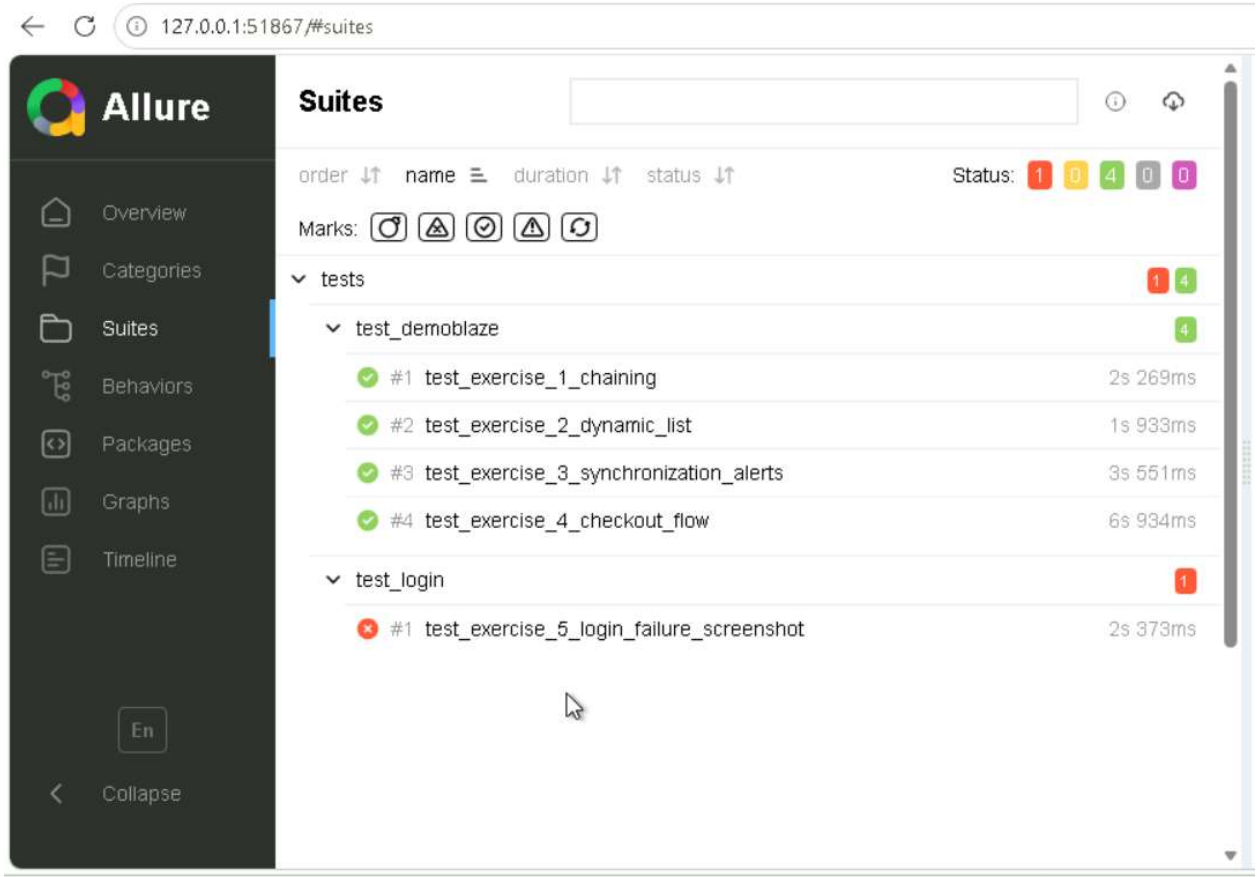
Login Failed Screenshot

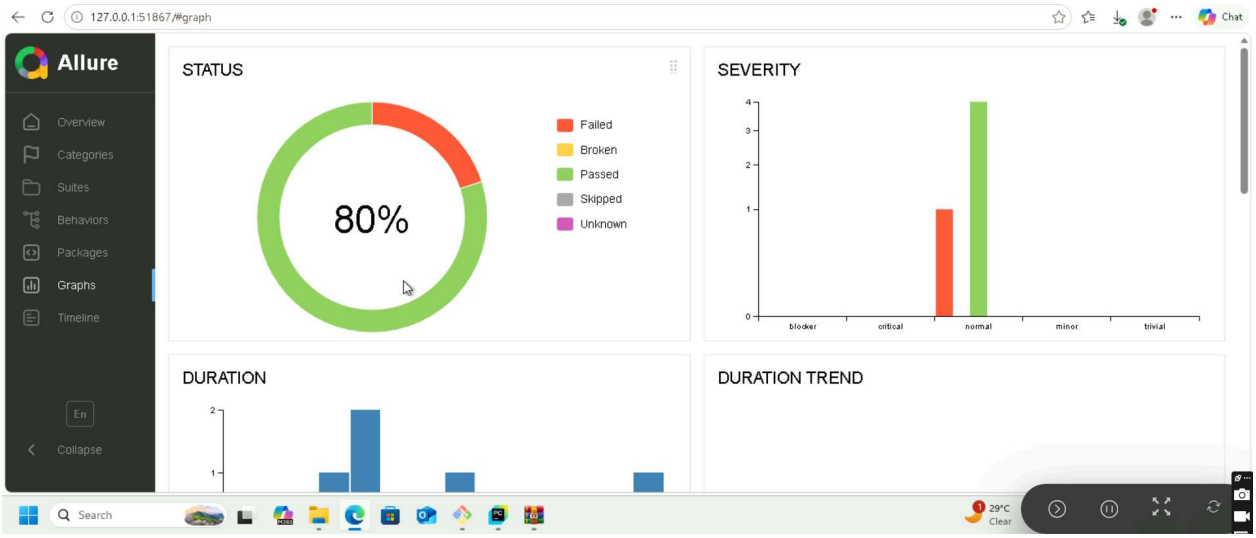


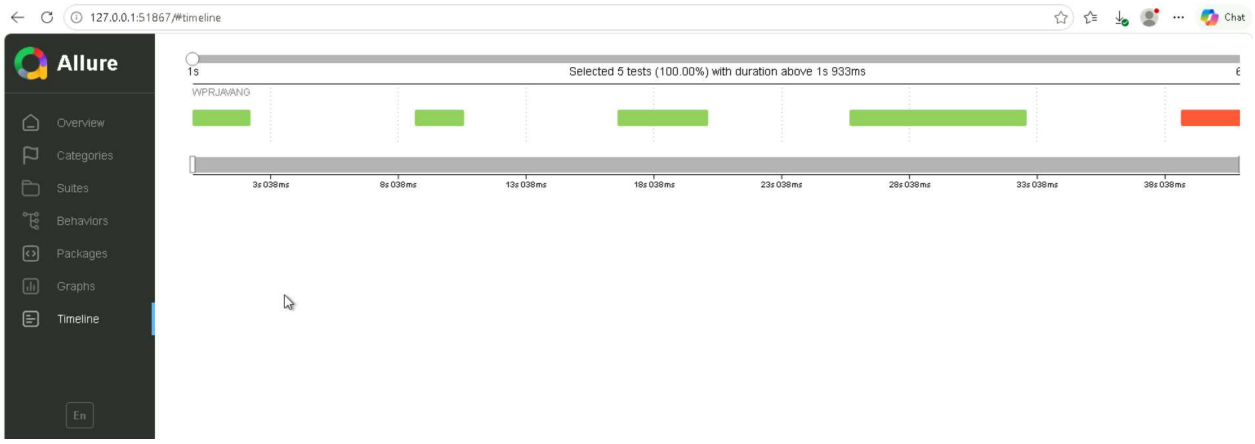
Reports




Allure
Overview
Categories
Suites
Behaviors
Packages
Graphs
Timeline
En
Collapse
Categories
order
name
duration
status
Status: 1 0 0 0 0
Marks:
Product defects 1
AssertionError: assert 'User does not exist.' == 'Please fill ...and Password.'
[0m[91m- Please fill out Username and Password.[39;49;00m[90m[39;49;00m
[92m+ User does not exist.[39;49;00m[90m[39;49;00m 1
#1 test_exercise_5_login_failure_screenshot 2s 373ms


127.0.0.1:51867/#suites
Allure
Overview
Categories
Suites
Behaviors
Packages
Graphs
Timeline
En
Collapse
Suites
order
name
duration
status
Status: 1 0 4 0 0
Marks:
tests 1 4
test_demoblaze 4
#1 test_exercise_1_chaining 2s 269ms
#2 test_exercise_2_dynamic_list 1s 933ms
#3 test_exercise_3_synchronization_alerts 3s 551ms
#4 test_exercise_4_checkout_flow 6s 934ms
test_login 1
#1 test_exercise_5_login_failure_screenshot 2s 373ms


127.0.0.1:51867/#graph
Chat
Allure
Overview
Categories
Suites
Behaviors
Packages
Graphs
Timeline
En
Collapse
STATUS
80%
Failed
Broken
Passed
Skipped
Unknown
SEVERITY
blocker
critical
normal
minor
trivial
DURATION
DURATION TREND
Search
29°C
Clear


127.0.0.1:51867/#timeline
Chat
Allure
Overview
Categories
Suites
Behaviors
Packages
Graphs
Timeline
En
1s
Selected 5 tests (100.00%) with duration above 1s 933ms
WPRJAVANG
3s 038ms
8s 038ms
13s 038ms
18s 038ms
23s 038ms
28s 038ms
33s 038ms
38s 038ms